Deliverable 2: Relational Database Design

Project: Research Lab Manager

Members: Rohit Deshmukh, Joao Christiansen, Manisha Dwivedi

---

1. Goals and Revisions

Goals for Phase 2

The primary goal of this phase was to map the conceptual Enhanced Entity-Relationship (EER) diagram created in Phase 1 into a logical Relational Schema. This involved:

- Identifying primary and foreign keys to maintain data integrity.
- Normalizing tables to reduce redundancy.
- Translating EER specializations (Lab Members) into a concrete table structure.
- Defining SQL Data Definition Language (DDL) to prepare for physical implementation.

Revisions to Phase 1

No revisions were made to the original Phase 1 EER diagram. The conceptual model was found to be sufficient and accurate for the requirements of the Research Lab Manager system; therefore, the mapping proceeded based on the initial design.

---

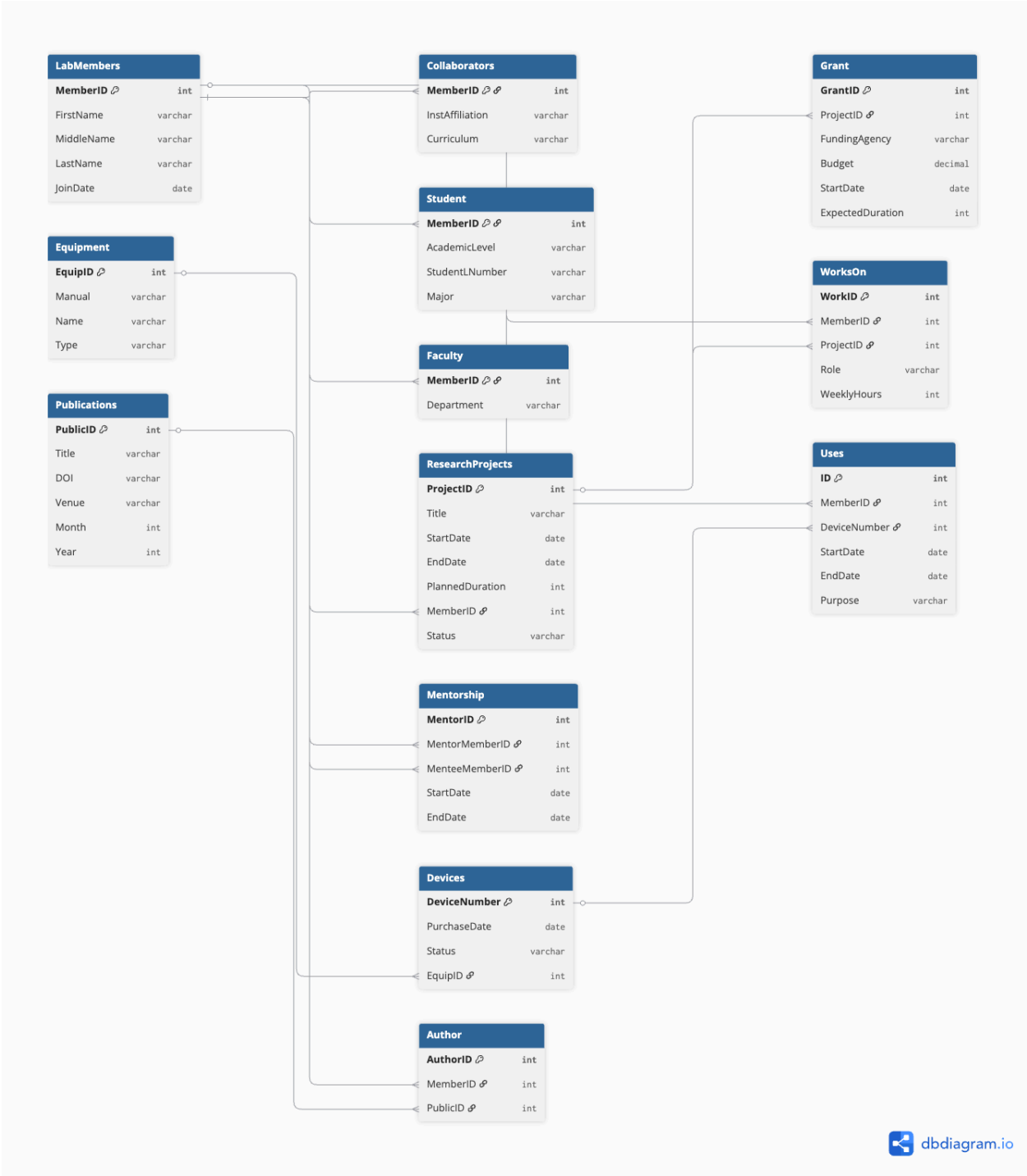
2. EER to Relational Mapping

Mapping Strategy

The mapping followed the standard EER-to-Relational algorithm:

- **Specialization (Lab Members):** We used the **Multiple Tables (Subclass and Superclass)** approach. A base table LabMembers contains shared attributes, while Student, Faculty, and Collaborators exist as separate tables linked via MemberID as both a Primary Key and a Foreign Key.
- **Binary M:N Relationships:** Relationships such as "WorksOn" (Members to Projects) and "Author" (Members to Publications) were mapped to separate relation tables to handle the many-to-many cardinality.
- **Weak Entities/Attributes:** Multi-valued or specific component attributes were handled via normalization to ensure each attribute contains atomic values.

Relational Schema Diagram



Constraints (Beyond Referential Integrity)

1. **Student:** StudentNumber must be unique and follow the institutional format (e.g., 'L' followed by 8 digits).
2. **ResearchProjects:** EndDate must be greater than or equal to StartDate.
3. **Publications:** Month must be an integer between 1 and 12.
4. **Mentorship:** MentorMemberID cannot be equal to MenteeMemberID (a member cannot mentor themselves).
5. **Grants:** Budget must be a positive decimal value (> 0).
6. **WorksOn:** WeeklyHours must be a non-negative integer.

---

3. SQL Table Creation (DDL)

```
CREATE TABLE LabMembers (
    MemberID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    MiddleName VARCHAR(50),
    LastName VARCHAR(50) NOT NULL,
    JoinDate DATE NOT NULL
);

CREATE TABLE Student (
    MemberID INT PRIMARY KEY,
    AcademicLevel VARCHAR(20),
    StudentNumber VARCHAR(20) UNIQUE NOT NULL,
    Major VARCHAR(50),
    FOREIGN KEY (MemberID) REFERENCES LabMembers(MemberID) ON DELETE
CASCADE
);

CREATE TABLE Faculty (
    MemberID INT PRIMARY KEY,
    Department VARCHAR(100),
    FOREIGN KEY (MemberID) REFERENCES LabMembers(MemberID) ON DELETE
CASCADE
);

CREATE TABLE Collaborators (
    MemberID INT PRIMARY KEY,
```

```sql
    InstAffiliation VARCHAR(100),
    Curriculum VARCHAR(255),
    FOREIGN KEY (MemberID) REFERENCES LabMembers(MemberID) ON DELETE
CASCADE
);

CREATE TABLE ResearchProjects (
    ProjectID INT PRIMARY KEY,
    Title VARCHAR(200) NOT NULL,
    StartDate DATE,
    EndDate DATE,
    PlannedDuration INT,
    MemberID INT,
    Status VARCHAR(20),
    FOREIGN KEY (MemberID) REFERENCES LabMembers(MemberID)
);

CREATE TABLE Grants (
    GrantID INT PRIMARY KEY,
    ProjectID INT,
    FundingAgency VARCHAR(100),
    Budget DECIMAL(15, 2),
    StartDate DATE,
    ExpectedDuration INT,
    FOREIGN KEY (ProjectID) REFERENCES ResearchProjects(ProjectID) ON
DELETE CASCADE
);

CREATE TABLE Equipment (
    EquipID INT PRIMARY KEY,
    Manual VARCHAR(255),
    Name VARCHAR(100) NOT NULL,
    Type VARCHAR(50)
);

CREATE TABLE Devices (
    DeviceNumber INT PRIMARY KEY,
    PurchaseDate DATE,
    Status VARCHAR(20),
    EquipID INT,
    FOREIGN KEY (EquipID) REFERENCES Equipment(EquipID) ON DELETE SET NULL
);
```

```sql
CREATE TABLE WorksOn (
    WorkID INT PRIMARY KEY,
    MemberID INT,
    ProjectID INT,
    Role VARCHAR(50),
    WeeklyHours INT,
    UNIQUE (MemberID, ProjectID),
    FOREIGN KEY (MemberID) REFERENCES LabMembers(MemberID),
    FOREIGN KEY (ProjectID) REFERENCES ResearchProjects(ProjectID)
);

CREATE TABLE Mentorship (
    MentorID INT PRIMARY KEY,
    MentorMemberID INT,
    MenteeMemberID INT,
    StartDate DATE,
    EndDate DATE,
    FOREIGN KEY (MentorMemberID) REFERENCES LabMembers(MemberID),
    FOREIGN KEY (MenteeMemberID) REFERENCES LabMembers(MemberID)
);

CREATE TABLE Uses (
    ID INT PRIMARY KEY,
    MemberID INT,
    DeviceNumber INT,
    StartDate DATE,
    EndDate DATE,
    Purpose VARCHAR(255),
    FOREIGN KEY (MemberID) REFERENCES LabMembers(MemberID),
    FOREIGN KEY (DeviceNumber) REFERENCES Devices(DeviceNumber)
);

CREATE TABLE Publications (
    PublicID INT PRIMARY KEY,
    Title VARCHAR(255) NOT NULL,
    DOI VARCHAR(100) UNIQUE,
    Venue VARCHAR(100),
    Month INT,
    Year INT
);

CREATE TABLE Author (
    AuthorID INT PRIMARY KEY,
```

```
    MemberID INT,
    PublicID INT,
    UNIQUE (MemberID, PublicID),
    FOREIGN KEY (MemberID) REFERENCES LabMembers(MemberID),
    FOREIGN KEY (PublicID) REFERENCES Publications(PublicID)
);
```

---

4. Design Justifications

- **Subclass Mapping:** We chose the multiple-table approach for LabMembers to enforce strict typing. This ensures that only Students have a StudentNumber, preventing data pollution in the Faculty or Collaborator categories.
- **Self-Referencing Relationship:** The Mentorship table was designed to link two different instances of LabMembers. This allows the system to track hierarchical growth within the lab without creating redundant entity types.
- **Separation of Equipment and Devices:** Equipment represents the "model" (e.g., "Oculus Rift"), while Devices represents the physical "units" (e.g., Unit #104). This allows for easier tracking of maintenance and status for individual items.

---

5. Challenges Encountered

- **Recursive Foreign Keys:** Defining the Mentorship table required careful naming of foreign keys to distinguish between the mentor and the mentee while they both reference the same parent table (LabMembers).
- **Execution Order:** Determining the correct order for the SQL CREATE TABLE scripts was challenging due to the high number of foreign key dependencies. Tables with no dependencies (like Equipment and Publications) had to be created first.
- **Managing M:N Uniqueness:** In tables like Author and WorksOn, we had to implement composite UNIQUE constraints to ensure a member isn't listed as an author or worker on the same item multiple times.